Lab 6: Asymmetric Encryption using RSA (Rivest–Shamir–Adleman)

Objective:

In this lab, you will learn the basics of public-key cryptography using the RSA algorithm. You will generate a pair of RSA keys, encrypt a message using the public key, and then decrypt the message using the private key. This will demonstrate how asymmetric encryption ensures both confidentiality and secure communication.

Overview:

- RSA is an asymmetric cryptographic algorithm that uses a pair of keys:

  - Public Key: Used to encrypt messages.

  - Private Key: Used to decrypt messages.

  With RSA, even if someone has access to the public key and the encrypted message, they cannot decrypt it without the corresponding private key.

Note: You can complete this lab either by creating a **single Python file** (`rsa_lab.py`) containing all steps sequentially, or by using separate scripts. If using separate scripts, make sure to hardcode/paste your Base64 encrypted string from Part 2 into your `encoded_encrypted_message` variable in Part 3.

Pre-requisites:

1. Python installed on your operating system.
2. `pycryptodome` library installed (Follow the system-specific command below).

Part 1: Key Generation

Step 1: Install the `pycryptodome` library

Open your terminal or command prompt and run the command matching your operating system:

- For Kali Linux / Debian Users: **sudo apt update && sudo apt install -y python3-pycryptodome**
- For Windows Users (Command Prompt / PowerShell): **pip install pycryptodome**

Step 1b: Verify Installation Verify the installation by running the test snippet for your OS:

- On Kali Linux: python3 -c "from Crypto.PublicKey import RSA; print('PyCryptodome loaded successfully!')"
- On Windows: python -c "from Crypto.PublicKey import RSA; print('PyCryptodome loaded successfully!')"

2. Use the following Python code to generate a pair of RSA keys.

```python
from Crypto.PublicKey import RSA

# Generate a 2048-bit RSA key pair (private and public key)
key = RSA.generate(2048)

# Export the private key and save it to a file
private_key = key.export_key()
with open('private_key.pem', 'wb') as f:
    f.write(private_key)

# Export the public key and save it to a file
public_key = key.publickey().export_key()
with open('public_key.pem', 'wb') as f:
    f.write(public_key)

print("RSA key pair generated and saved")
```

Part 2: RSA Encryption

Now that you have a public key, let's use it to encrypt a message.

Steps:

1. Load the public key and use it to encrypt a plaintext message.

2. The `PKCS1_OAEP` cipher is used to pad the message before encryption.

```python
from Crypto.Cipher import PKCS1_OAEP
from Crypto.PublicKey import RSA
from base64 import b64encode

# Load the public key from the file
with open('public_key.pem', 'rb') as f:
    public_key = RSA.import_key(f.read())

# Encrypt the message
message = "Hello, welcome to the world of RSA encryption!"
cipher = PKCS1_OAEP.new(public_key)
encrypted_message = cipher.encrypt(message.encode())

# Encode the encrypted message in base64 for easier display
encoded_encrypted_message = b64encode(encrypted_message).decode()

print("Encrypted Message:", encoded_encrypted_message)
```

Part 3: RSA Decryption

To decrypt the message, the recipient will use their private key.

Steps:

1. Load the private key.

2. Decrypt the previously encrypted message.

```python
from Crypto.Cipher import PKCS1_OAEP
from Crypto.PublicKey import RSA
from base64 import b64decode

# Load the private key
with open('private_key.pem', 'rb') as f:
    private_key = RSA.import_key(f.read())

# Decode base64 and decrypt with private key
decoded_encrypted_message = b64decode(encoded_encrypted_message)
cipher = PKCS1_OAEP.new(private_key)
decrypted_message = cipher.decrypt(decoded_encrypted_message).decode()

print("Decrypted Message:", decrypted_message)
```

Part 4: Discussion Questions

1. What are the advantages and disadvantages of RSA compared to symmetric encryption (e.g., AES)?

_____
_____
_____
_____
_____

2. What happens if the private key is compromised?

_____
_____
_____
_____
_____
_____

3. How could you prove non-repudiation?

Sign a Message

- **Objective:** In this challenge, you will simulate how RSA can be used for digital signatures to ensure both integrity and authenticity of messages.

1. Use the private key to "sign" a message by encrypting its hash.

2. The recipient can then use the sender's public key to verify the signature by comparing it with the message's hash.

```
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

# Sign the message (private key holder)
message = "This is a very important message."
hash_message = SHA256.new(message.encode())
signature = pkcs1_15.new(private_key).sign(hash_message)

# Verify the message (public key holder)
try:
    pkcs1_15.new(public_key).verify(hash_message, signature)
    print("The signature is valid.")
except (ValueError, TypeError):
    print("The signature is not valid.")
```